BeaconKit 아키텍처 분석 - 08. 저장소 아키텍처

1. 저장소 아키텍처 개요

BeaconKit은 **다층 저장소 시스템**을 사용하여 다양한 유형의 데이터를 효율적으로 관리합니다. 각 저장소는 특정 목적에 최적화되어 있으며, 성능과 데이터 무결성을 보장합니다.

1.1 저장소 계층 구조

```
┌─────────────────────────────────────────────────────────┐
│ StorageBackend │
│ (통합 저장소 인터페이스) │
├──────────────────┬──────────────────┬──────────────────────┤
│ BeaconDB │ BlockStore │ AvailabilityStore │
│ (상태 저장) │ (블록 메타데이터)│ (블롭 저장) │
├──────────────────┼──────────────────┼──────────────────────┤
│ DepositStore │ FileDB │ KVStore │
│ (예치 이벤트) │ (범용 파일 DB) │ (키-값 저장소) │
└──────────────────┴──────────────────┴──────────────────────┘
```

1.2 저장소 유형별 특징

BeaconDB: 비콘 체인 상태 데이터

- 검증자 정보, 잔액, 상태 루트

- 포크 인식 쿼리 지원
- 컨텍스트 기반 상태 관리

BlockStore: 블록 메타데이터 캐시

- 블록 루트, 타임스탬프, 상태 루트 매핑
- LRU 캐시 기반 메모리 저장
- 빠른 조회를 위한 인덱싱

AvailabilityStore: 블롭 사이드카 저장

- TTL 기반 자동 정리
- DA 기간 내 데이터 가용성 보장
- KZG 증명과 함께 저장

DepositStore: 예치 계약 이벤트

- 실행 레이어 동기화
- 머클 트리 관리
- 증분 업데이트 지원

2. StorageBackend 인터페이스

2.1 통합 인터페이스

```
type StorageBackend interface {
// 각 저장소에 대한 접근자
AvailabilityStore() AvailabilityStore
BlockStore() BlockStore
DepositStore() DepositStore
```

```
// 컨텍스트 기반 상태 접근
StateFromContext(ctx context.Context) BeaconState
}
```

2.2 Backend 구현

```
type Backend struct {
// 각 저장소 인스턴스
availabilityStore AvailabilityStore
blockStore BlockStore
depositStore DepositStore
beaconDB BeaconDB
// 상태 관리
stateManager StateManager
}

func (b *Backend) StateFromContext(ctx context.Context) BeaconState {
// 컨텍스트에서 포크 정보 추출
// 해당 포크에 맞는 상태 반환
return  b.stateManager.GetStateForContext(ctx)
}
```

3. BeaconDB - 상태 저장소

3.1 BeaconDB 구조

```go
type KVStore struct {
// 기본 키-값 저장소
kvStoreService KVStoreService
// 인덱스 관리자들
validatorIndex *index.ValidatorIndex
balanceIndex *index.BalanceIndex
stateRootIndex *index.StateRootIndex
// 캐시 레이어
validatorCache *lru.Cache[ValidatorIndex, *Validator]
balanceCache *lru.Cache[ValidatorIndex, math.Gwei]
}
```

3.2 주요 기능

검증자 관리:

```go
// 검증자 정보 저장/조회
func (kv *KVStore) SetValidator(index ValidatorIndex, validator *Validator) erro
func (kv *KVStore) GetValidator(index ValidatorIndex) (*Validator, error)
func (kv *KVStore) GetValidatorByPubkey(pubkey BLSPubkey) (*Validator, error)
```

```go
// 검증자 상태 업데이트
func (kv *KVStore) UpdateValidatorStatus(index ValidatorIndex, status ValidatorS
```

잔액 관리:

```go
// 검증자 잔액 관리
func (kv *KVStore) SetBalance(index ValidatorIndex, balance math.Gwei) error
func (kv *KVStore) GetBalance(index ValidatorIndex) (math.Gwei, error)
func (kv *KVStore) GetTotalActiveBalance() (math.Gwei, error)
```

상태 루트 관리:

```go
// 상태 루트 히스토리
func (kv *KVStore) SetStateRoot(slot math.Slot, root common.Root) error
func (kv *KVStore) GetStateRoot(slot math.Slot) (common.Root, error)
```

3.3 포크 인식 쿼리

```go
// 컨텍스트 기반 포크 버전 결정
func (kv *KVStore) GetValidatorForContext(ctx context.Context, index ValidatorIn
forkVersion := extractForkVersionFromContext(ctx)
return  kv.getValidatorWithFork(index, forkVersion)
}
```

4. BlockStore - 블록 메타데이터 저장소

4.1 BlockStore 구조

```
type KVStore[BeaconBlockT BeaconBlock] struct {
// 활성화 여부 (가용성 윈도우가 0이면 비활성화)
enabled  bool

// LRU 캐시들 (단사 매핑 보장)
blockRoots *lru.Cache[common.Root, math.Slot] // 블록 루트 → 슬롯
timestamps *lru.Cache[math.U64, math.Slot] // 타임스탬프 → 슬롯
stateRoots *lru.Cache[common.Root, math.Slot] // 상태 루트 → 슬롯

logger log.Logger
}
```

4.2 주요 기능

블록 저장:

```
func (kv *KVStore[BeaconBlockT]) Set(blk BeaconBlockT) error {
if !kv.enabled {
kv.logger.Debug("skipping set because block store is not enabled")
return  nil
}
```

```
slot := blk.GetSlot()
kv.blockRoots.Add(blk.HashTreeRoot(), slot)
kv.timestamps.Add(blk.GetTimestamp(), slot)
kv.stateRoots.Add(blk.GetStateRoot(), slot)
return nil
}
```

조회 기능:

```
// 블록 루트로 슬롯 조회
func (kv *KVStore[BeaconBlockT]) GetSlotByBlockRoot(blockRoot common.Root) (math
if !kv.enabled {
return math.Slot(0), ErrBlockStoreNotEnabled
}

slot, ok := kv.blockRoots.Peek(blockRoot)
if !ok {
return 0, fmt.Errorf("slot not found at block root: %s", blockRoot)
}
return slot, nil
}
```

```go
// 타임스탬프로 부모 슬롯 조회
func (kv *KVStore[BeaconBlockT]) GetParentSlotByTimestamp(timestamp math.U64) (m
slot, ok := kv.timestamps.Peek(timestamp)
if !ok {
return  slot, fmt.Errorf("slot not found at timestamp: %d", timestamp)
}
if  slot == 0 {
return  slot, errors.New("parent slot not supported for genesis slot 0")
}
return  slot - 1, nil
}


// 상태 루트로 슬롯 조회
func (kv *KVStore[BeaconBlockT]) GetSlotByStateRoot(stateRoot common.Root) (math
slot, ok := kv.stateRoots.Peek(stateRoot)
if !ok {
return  0, fmt.Errorf("slot not found at state root: %s", stateRoot)
}
return  slot, nil
}
```

4.3 가용성 윈도우

설정 기반 활성화:

```
func NewStore[BeaconBlockT BeaconBlock](
logger log.Logger,
availabilityWindow int,
) *KVStore[BeaconBlockT] {
kvStore := &KVStore[BeaconBlockT]{
enabled: availabilityWindow != 0, // 0이면 비활성화
logger: logger,
}
if !kvStore.enabled {
return kvStore // 캐시 생성 없이 반환
}
// LRU 캐시들 생성 (availabilityWindow 크기)
kvStore.blockRoots, _ = lru.New[common.Root, math.Slot](availabilityWindow)
kvStore.timestamps, _ = lru.New[math.U64, math.Slot](availabilityWindow)
kvStore.stateRoots, _ = lru.New[common.Root, math.Slot](availabilityWindow)
return kvStore
}
```

5. AvailabilityStore - 블롭 저장소

5.1 데이터 가용성 개념

DA 기간 (Data Availability Period):

- 블롭 데이터가 네트워크에서 사용 가능해야 하는 기간
- 일반적으로 4096 에포크 (약 18일)
- 기간 경과 후 자동 정리

5.2 AvailabilityStore 기능

블롭 저장:

```
func (as *Store) Set(slot math.Slot, sidecars BlobSidecars) error {
// 슬롯별로 블롭 사이드카 저장
// KZG 커밋먼트와 증명 포함
// TTL 설정으로 자동 만료
}
```

데이터 가용성 확인:

```
func (as *Store) IsDataAvailable(
ctx context.Context,
slot math.Slot,
body BeaconBlockBody,
) bool {
// 블록 바디의 모든 KZG 커밋먼트에 대해
// 해당하는 블롭 사이드카가 존재하는지 확인
commitments := body.GetBlobKzgCommitments()
for  i, commitment := range  commitments {
```

```
if !as.HasBlobSidecar(slot, uint64(i), commitment) {
return false
}
}
return true
}
```

자동 정리:

```
func (as *Store) Prune(currentSlot math.Slot) error {
// DA 기간을 벗어난 블롭들 정리
cutoffSlot := currentSlot - as.chainSpec.MinEpochsForBlobSidecarsRequest() * as.
return  as.deleteBeforeSlot(cutoffSlot)
}
```

6. DepositStore - 예치 저장소

6.1 예치 동기화

실행 레이어 모니터링:

```
type Store struct {
// 예치 계약 주소
depositContract common.Address
```

```
// 마지막 동기화된 블록
lastSyncedBlock uint64
// 예치 이벤트 저장소
deposits map[uint64]*Deposit // 인덱스 → 예치
// 머클 트리 관리
merkleTree *MerkleTree
}
```

증분 동기화:

```
func (ds *Store) SyncToBlock(blockNumber uint64) error {
// lastSyncedBlock부터 blockNumber까지의 예치 이벤트 조회
events, err := ds.fetchDepositEvents(ds.lastSyncedBlock+1, blockNumber)
if err != nil {
return err
}
// 각 이벤트를 저장소에 추가
for _, event := range events {
if err := ds.addDeposit(event); err != nil {
return err
}
}
ds.lastSyncedBlock = blockNumber
return nil
```

```
}
```

6.2 머클 트리 관리

예치 루트 계산:

```
func (ds *Store) GetDepositRoot() common.Root {
return ds.merkleTree.Root()
}

func (ds *Store) GetDepositsByIndex(
startIndex, endIndex uint64,
) ([]*Deposit, common.Root, error) {
deposits := make([]*Deposit, 0, endIndex-startIndex)
for i := startIndex; i < endIndex; i++ {
if deposit, exists := ds.deposits[i]; exists {
deposits = append(deposits, deposit)
}
}
return deposits, ds.GetDepositRoot(), nil
}
```

7. FileDB - 범용 파일 데이터베이스

7.1 FileDB 구조

```go
type DB struct {
// 파일 시스템 기반 저장소
dataDir  string
fileSystem afero.Fs
// 압축 및 인코딩
compression CompressionType
encoding EncodingType
// 동시성 제어
mu sync.RWMutex
}
```

7.2 범위 쿼리 지원

```go
type RangeDB struct {
*DB
// 범위 쿼리 최적화
indexCache *lru.Cache[string, []byte]
}

func (rdb *RangeDB) GetRange(start, end []byte) ([]KVPair, error) {
// 시작과 끝 키 사이의 모든 키-값 쌍 반환
```

```
// 인덱스 캐시를 활용한 최적화

}
```

8. 성능 최적화

8.1 캐싱 전략

다층 캐싱:

```
// L1: 메모리 캐시 (LRU)

validatorCache *lru.Cache[ValidatorIndex, *Validator]



// L2: 디스크 캐시 (압축)

diskCache *filedb.DB



// L3: 영구 저장소 (데이터베이스)

persistentStore  database.DB
```

캐시 무효화:

```
func (kv *KVStore) InvalidateCache(index ValidatorIndex) {

kv.validatorCache.Remove(index)

kv.balanceCache.Remove(index)

}
```

8.2 배치 처리

배치 업데이트:

```
func (kv *KVStore) BatchUpdate(updates []Update) error {
batch := kv.kvStoreService.NewBatch()
for _, update := range updates {
batch.Set(update.Key, update.Value)
}
return batch.Commit()
}
```

8.3 압축 및 인코딩

SSZ 직렬화:

```
func (kv *KVStore) encodeValidator(validator *Validator) ([]byte, error) {
return validator.MarshalSSZ()
}

func (kv *KVStore) decodeValidator(data []byte) (*Validator, error) {
validator := &Validator{}
return validator, validator.UnmarshalSSZ(data)
}
```

9. 데이터 무결성

9.1 체크섬 검증

```
func (db *DB) verifyChecksum(key []byte, data []byte) error {
expectedChecksum := computeChecksum(data)
storedChecksum := db.getChecksum(key)
if !bytes.Equal(expectedChecksum, storedChecksum) {
return ErrChecksumMismatch
}
return nil
}
```

9.2 원자적 업데이트

```
func (kv *KVStore) AtomicUpdate(fn func(tx Transaction) error) error {
tx := kv.kvStoreService.BeginTransaction()
defer tx.Rollback()
if err := fn(tx); err != nil {
return err
}
return tx.Commit()
}
```

10. 모니터링 및 메트릭

10.1 저장소 메트릭

```
type StorageMetrics struct {
// 읽기/쓰기 성능
readLatency prometheus.Histogram
writeLatency prometheus.Histogram
// 캐시 성능
cacheHitRate prometheus.Gauge
cacheMissRate prometheus.Gauge
// 저장소 사용량
diskUsage prometheus.Gauge
memoryUsage prometheus.Gauge
}
```

10.2 상태 추적

저장소 상태:

- 디스크 사용량
- 메모리 사용량
- 캐시 적중률
- 읽기/쓰기 지연시간

데이터 무결성:

- 체크섬 오류 수
- 손상된 데이터 감지
- 복구 작업 수행

11. 설계 원칙

11.1 계층화

- 각 저장소는 특정 목적에 최적화
- 명확한 인터페이스 분리
- 독립적인 성능 튜닝

11.2 확장성

- 수평적 확장 지원
- 샤딩 가능한 설계
- 부하 분산 고려

11.3 안정성

- 데이터 무결성 보장
- 원자적 업데이트
- 장애 복구 메커니즘

11.4 성능

- 다층 캐싱
- 배치 처리
- 압축 및 최적화

이 저장소 아키텍처는 BeaconKit이 다양한 유형의 데이터를 효율적이고 안전하게 관리하면서도 높은 성능을 유지할 수 있게 해주는 핵심 인프라입니다.